

Computer Security

Chapter 4

Side-Channel Attacks

This chapter was generated using AI from course materials,
and subsequently reviewed by Roberto Guanciale

Contents

1	Why Correct Functionality Is Not Enough	2
2	Information Flow and Noninterference	2
3	Timing Leaks in PIN Checking and Exponentiation	3
4	Traffic Analysis and Browser Fingerprinting	5
5	Caches as Shared, Observable State	7
6	Types of Cache Attacks	8
7	An Access-Driven Attack on Table-Based AES	9
8	Worked Toy Example: Narrowing a Secret with Cache Lines	10
9	Countermeasures and Their Limits	11
10	Exercises	13

1 Why Correct Functionality Is Not Enough

A security mechanism may compute the intended result and still reveal a secret through the way it executes. An encryption implementation, for example, might return only a ciphertext while its execution time or memory accesses depend on the key. An attacker who can observe these effects gains information beyond the intended output. This is a **side channel**.

An implementation can become the weak link in several ways. A software bug, such as a buffer overflow, can violate the program's intended memory boundaries. A fault-injection attack deliberately perturbs execution; Rowhammer is one example. A side-channel attack instead infers secrets from observable effects of execution, without requiring the victim to compute an incorrect result.

The description of side channels as passive and non-invasive concerns the observation of leakage. It does not mean that every attack leaves the environment untouched: an attacker may choose requests, fill a cache, or evict data before observing how the victim uses it.

Observations can include the victim's running time, the attacker's own running time when competing for a shared resource, power consumption, electromagnetic radiation, sound, temperature, or sensor measurements.

2 Information Flow and Noninterference

2.1 High and low information

Information-flow security classifies values according to who may observe them. Here, **high** means secret and **low** means public. Public inputs and observable outputs are both low. The security question is whether changing an initial secret can change an observable result while the public inputs remain fixed.

```
int secret;           // high
int public, output;   // low
```

The assignment `output = public + 1` depends only on public information and reveals nothing about the secret. In contrast, `output = secret + 1` creates an **explicit flow**: the output directly depends on a secret value.

A variable's name or storage location does not by itself determine whether an execution reveals an initial secret. Consider the complete sequence

```
secret = public + 1;
output = secret;
```

The first assignment overwrites the original secret with a value determined entirely by the public input. The second assignment therefore outputs `public + 1`, regardless of the original secret. The sequence does not leak the initial high value.

A dependency can also arise through control flow:

```
if (secret > 0)
    output = 1;
```

Suppose `output` initially equals zero. Its final value reveals whether the condition was true. This is an **implicit flow**; assigning a constant does not prevent a leak when a secret controls whether the assignment executes.

2.2 Comparing two executions

A state contains the program's variable values. Write $s_1 \sim_L s_2$ when the two states agree on all low values, including public inputs and outputs, although their high values may differ. For a deterministic terminating program P , the basic noninterference condition is

$$s_1 \sim_L s_2 \implies P(s_1) \sim_L P(s_2).$$

Thus, two executions starting from indistinguishable public states must end in indistinguishable public states. The attacker should not learn which secret state was used by inspecting the final low values.

2.3 Declassification: allowing a specific release

Strict noninterference is too strong for many useful programs. A PIN checker must reveal whether the supplied PIN is correct:

```
output = (input == pin);
```

Here `input` and `output` are public, and `pin` is secret. The intended policy permits the equality result to be disclosed, but does not permit disclosure of the entire PIN. This controlled release is called **declassification**.

Let $D(s)$ be the information the policy permits the program to release. A precise condition for this example is

$$(s_1 \sim_L s_2 \wedge D(s_1) = D(s_2)) \implies P(s_1) \sim_L P(s_2),$$

where $D(s) = (s.\text{input} = s.\text{pin})$. Only executions that agree on both public inputs and the permitted release need to remain indistinguishable. The conjunction is essential: the policy permits public outputs to differ when the authorized equality result differs. Requiring indistinguishable outputs for all low-equivalent initial states would prohibit this release.

3 Timing Leaks in PIN Checking and Exponentiation

3.1 Extending the observation model

Let $O(P, s)$ denote everything an attacker can observe during an execution, including its timing or cache footprint. Even if the final public values of two executions are equal, the observations may differ:

$$P(s_1) \sim_L P(s_2), \quad O(P, s_1) \neq O(P, s_2).$$

For example, two incorrect PIN guesses both return false, but one may take longer because it matches a longer prefix. The timing releases information beyond the declassified success/failure bit.

3.2 Early-exit PIN comparison

Consider a four-digit decimal PIN, including leading zeroes:

```
check_pin(candidate):
    for i in 0..3:
        if candidate[i] != stored_pin[i]:
            return False
    return True
```

The returned Boolean is a public output: it deliberately reveals whether the complete candidate matches. The number of comparisons performed before the return is not part of that intended release.

There are $10^4 = 10,000$ possible PINs. For a uniformly distributed PIN, testing distinct complete guesses in a fixed order takes 5,000 attempts on average. Three distinct guesses succeed with probability $3/10,000$.

The early return changes this picture if timing is measurable. A mismatch in the first position performs one comparison, while a mismatch in the second performs two. The attacker can therefore learn whether the first digit is correct even when the complete candidate is rejected.

3.3 Extracting the first PIN digit from timing

The observation that makes the attack possible. Write the stored PIN as $p_0p_1p_2p_3$, where each p_i is a decimal digit. Choose a fixed three-digit suffix, such as 000, and submit candidates of the form $d000$ for $d \in \{0, 1, \dots, 9\}$. The suffix is only filler: the attacker does not need to know whether any of its digits are correct.

If $d \neq p_0$, the checker compares the first digit, detects a mismatch, and immediately returns false. If $d = p_0$, it continues to the second position and performs at least one extra comparison. Matching more of the suffix can only make the checker proceed further. Thus, in an ideal model where each additional comparison takes positive time and other costs are fixed, the correct first digit produces the longest execution among the ten candidates.

Let $N(d)$ be the number of digit comparisons performed on candidate $d000$. Then

$$N(d) = 1 \quad \text{if } d \neq p_0, \quad N(p_0) \geq 2.$$

A simple timing model is

$$T(d) = t_{\text{fixed}} + N(d) \Delta + \epsilon,$$

where t_{fixed} includes overhead common to all candidates, $\Delta > 0$ is the time per comparison, and ϵ represents measurement noise. The attack seeks the extra work hidden inside the response time, not a different Boolean result. This model describes the vulnerable early-exit implementation; it is not a guarantee that every real system exposes a measurable difference.

Differential timing attack. Submit two candidates $a000$ and $b000$, where $a \neq b$, and compare their execution times through the difference $D = T(a000) - T(b000)$. Holding the suffix fixed isolates the effect of the first digit, while subtraction cancels the fixed overhead in the ideal timing model. If $D > 0$, candidate $a000$ performed more comparisons, so a is the correct first digit. If $D < 0$, b is correct. If $D = 0$, both first digits are wrong in the noiseless model: distinct digits cannot both be correct, and either correct digit would cause extra work.

For example, with stored PIN 5729, comparing 4000 with 5000 gives a negative difference: the first candidate stops after one comparison, whereas the second performs two. The attacker therefore infers 5 without knowing the remaining digits. If both candidates have the same ideal time, try another pair of digits. In practice, repeat paired measurements, alternate their order, and look for a consistent difference; a single noisy sample or an approximate tie is not enough to establish which digit is correct.

3.4 Square-and-multiply

RSA uses modular exponentiation for both public-key and private-key operations. For example, the core private-key operation in RSA decryption computes $c^{d_{\text{RSA}}} \bmod n$, where c is the ciphertext representative, n is the public modulus, and d_{RSA} is the secret private exponent. In the generic computation $b^e \bmod m$ below, this corresponds to $b = c$, $m = n$, and $e = d_{\text{RSA}}$. Thus the algorithm's exponent is secret in this use case. The symbol e here is a generic parameter; conventional RSA notation instead uses e_{RSA} for the *public* exponent.

If square-and-multiply implements this private-key operation, its conditional multiplications depend on bits of the secret exponent. Timing observations may therefore reveal information about private-key material even though the modular-exponentiation result is correct. A public-exponent operation does not have the same confidentiality concern for its exponent, because those bits are already public.

Modular exponentiation computes $b^e \bmod m$. A square-and-multiply implementation has the form

```

if modulus == 1:
    return 0
result = 1
base = base mod modulus
while exponent > 0:
    if exponent mod 2 == 1:
        result = (result * base) mod modulus
    exponent = exponent >> 1
    base = (base * base) mod modulus
return result

```

Each loop iteration processes one exponent bit. Squaring always occurs, but the multiplication into **result** occurs only for a one-bit. When the exponent is secret, this conditional work may reveal information about its bits. Aggregate time can reveal information about the number of one-bits; a more detailed trace can distinguish individual operations.

4 Traffic Analysis and Browser Fingerprinting

A side channel need not arise inside a processor. Network activity and browser behavior expose measurements that can reveal what a user is doing or link otherwise separate interactions. Encryption protects message contents, but packet sizes, directions, and timing may remain visible.

4.1 Video fingerprinting

A video stream protected by HTTPS or TLS can still reveal which video is being watched. Different videos produce different delivery patterns: scene complexity affects compressed data sizes, while segment boundaries, buffering, and bitrate selection shape the resulting bursts of traffic.

The intuition can be seen by comparing two very different kinds of film content. The first image below comes from **2001: A Space Odyssey**, whose long shots can contain a nearly black background for several minutes. The second comes from **Star Wars**, whose action scenes contain rapid motion, explosions, and many changing details.



Figure 1: Two frames with different visual complexity: a dark, slowly changing shot and a rapidly changing action scene.

Video codecs exploit temporal and spatial redundancy. A dark, nearly static scene can often be represented with a small number of bits: neighboring pixels are similar, and successive frames differ little. A fast-paced action scene contains more edges, motion, and changes between frames, so the codec may need more residual information and therefore more bits. Even when the payload is encrypted, these differences can remain visible indirectly through segment sizes, packet bursts, and the gaps between bursts. A sequence of such features can become a fingerprint for the video being streamed.

An observer can measure packet sizes, packet directions, inter-arrival times, burst lengths, and gaps between bursts. These measurements form a trace without requiring access to decrypted video data. A typical inference process is:

1. Collect labeled traffic traces while known videos are played.
2. Extract features describing the size and timing patterns of each trace.
3. Train a classifier, such as a neural network, to associate those patterns with video identities.
4. Apply the classifier to an observed stream to estimate which video is being watched.

The attack relies on delivery patterns being sufficiently distinctive and repeatable. Network conditions, adaptive bitrate changes, and unrelated traffic can reduce accuracy. A classifier trained on a particular collection of videos does not automatically identify every possible stream.

Countermeasures target the distinguishing features. Packet padding can hide size differences, although padding individual packets alone may leave burst sizes visible. Making segment sizes and delivery patterns more uniform reduces their distinctiveness. Mixing several users' traffic can make it harder to attribute a pattern to one stream. These defenses trade bandwidth, latency, or system complexity for less informative observations.

An example of encrypted-video identification is *Beauty and the Burst: Remote Identification of Encrypted Video Streams* (USENIX Security, 2017).

4.2 SSH keystroke recovery

Interactive SSH can transmit a small packet promptly after each key press. When this occurs, inter-packet delays approximate the user's inter-keystroke delays. Packet sizes and directions provide additional context. Encryption conceals the character values but does not necessarily conceal when they were typed.

Typing intervals are not independent of the characters. Moving between two particular keys may take a different amount of time from moving between another pair; surrounding characters and the typist's habits also affect the delay. An attacker can first observe known typing sequences and learn probability distributions for character-pair timings. For an unknown sequence, the attacker measures inter-arrival times, estimates likely pairs, and combines compatible pairs into candidate words or passwords.

The result is a probabilistic ranking of candidate strings, rather than direct decryption of packets. The classical study *Timing Analysis of Keystrokes and Timing Attacks on SSH* (USENIX Security, 2001) demonstrates how statistical models can use this signal. Actual leakage depends on buffering, network noise, and whether the relevant input is sent one character at a time.

Sending packets at a constant rate reduces the correspondence between typing and transmission. To hide pauses as well as character timing, a scheme must maintain the schedule even when there is no new keystroke, for example by sending cover traffic. This introduces bandwidth and responsiveness costs.

4.3 Browser fingerprinting

Browser fingerprinting identifies a browser without relying on cookies. A tracker combines client properties into a feature vector, including the user agent and browser version, operating system, screen dimensions, language, timezone, locale, installed fonts, and rendering behavior. Individually common attributes may form a distinctive combination.

Some properties can be read directly; others require an active measurement. For example, JavaScript can draw text or shapes on a canvas and hash the resulting pixels. Differences in fonts

and rendering can affect the result. Timing a rendering operation provides another measurable feature. Thus a fingerprint may combine reported configuration with observed behavior.

The tracker can use the resulting vector to estimate whether two visits came from the same browser or whether several accounts are operated from the same device. These are inferences: matching fingerprints do not prove that the same person was involved, and a browser’s fingerprint may change over time. The identifying potential of configuration combinations is examined in *How Unique Is Your Web Browser?* (PETS, 2010).

Two defensive approaches are **standardization** and **emulation**. Standardization makes many clients expose the same values and behavior, reducing the number of distinguishable groups. Emulation presents a controlled profile instead of the underlying device’s characteristics. Such a profile must remain consistent across the different features being measured; unusual or conflicting values can themselves become identifying information. Clearing cookies alone does not remove these observable properties.

5 Caches as Shared, Observable State

5.1 The memory hierarchy

Processors use caches to bridge the speed difference between computation and main memory. A cache holds a small selection of memory blocks so that repeated accesses can be served quickly. Multiple programs may share its capacity, either by running on the same processor at different times or by accessing shared cache resources concurrently.

A **cache hit** finds the requested block in the cache. A **cache miss** requires fetching it from a slower level. Fetching generally brings in a whole **cache line**, rather than just the requested byte. A cached block may be **evicted** to make room for another one. In a write-back cache, a modified or dirty block must eventually be written to the next level; a write-through policy instead propagates writes as they occur.

Caches are transparent to ordinary program semantics: a load should return the same value whether it hits or misses. They are not transparent to timing. Shared cache state therefore creates a way for one process to learn something about another process’s memory accesses.

5.2 A direct-mapped cache model

Consider a 4 KiB cache with 64-byte lines. In a direct-mapped model there are $4096/64 = 64$ slots, numbered 0 through 63, and address a maps to

$$\text{slot}(a) = \left\lfloor \frac{a}{64} \right\rfloor \bmod 64.$$

The first block covers byte addresses 0 through 63; the next covers 64 through 127. Addresses 0, 4096, and 8192 all map to slot 0. Accessing one after another therefore replaces the previous block in that slot. Two different addresses can **collide** even though their underlying memory is unrelated.

Block start address	Cache slot	Relation
0	0	First block
64	1	Next block
4096 (4 KiB)	0	Conflicts with address 0
4160 (4 KiB + 64)	1	Conflicts with address 64
8192 (8 KiB)	0	Conflicts with address 4096

Real caches may be set-associative, allowing several blocks per set. Their indexing, replacement, and sharing rules also vary. The examples below use the simple model when stated; their exact

addresses are not universal hardware facts. An attacker only needs contention for observable cache resources, not permission to read the victim’s memory.

6 Types of Cache Attacks

6.1 Time-driven attacks: measuring the victim

A time-driven attacker triggers a victim operation and measures its total time. For example:

```
victim():
    v1 = M[4096]
    v2 = M[4096 + 64 * secret]
    save(v1 + v2)
    return 0
```

Assume the relevant blocks are initially absent, the accesses execute as shown, and *M* denotes byte-addressed memory. If *secret* is zero, the second access hits the block brought in by the first. Otherwise it accesses a different block, potentially adding a miss. A time difference may therefore reveal whether the secret is zero. Initial cache state, prefetching, and other memory accesses matter to this inference.

6.2 Access-driven attacks: measuring the attacker

An access-driven attacker is another program sharing cache resources with the victim. The attacker infers which resources the victim used by measuring its own memory accesses. It does not need permission to read the victim’s data.

The **prime-and-probe** technique uses addresses that contend for the same cache slots or sets:

1. **Prime:** access attacker-controlled blocks to populate the cache.
2. **Victim execution:** allow the victim to access secret-dependent blocks.
3. **Probe:** time accesses to the primed blocks to detect evictions.

In the direct-mapped example, the attacker primes address 8192. A victim access to address 4096 evicts it, so probing 8192 afterwards is slow. In contrast, a victim access to 4160 uses slot 1. The victim’s load from $4096 + 64 * \text{secret}$ therefore influences which attacker access becomes slow.

A fuller observation primes and probes all 64 slots:

```
attacker_prime():
    for i in 0..63:
        read M[8192 + 64*i]

attacker_probe():
    for i in 0..63:
        start = time()
        read M[8192 + 64*i]
        elapsed[i] = time() - start
```

The slow entries in *elapsed* suggest which primed blocks were evicted. The timing includes measurement overhead and interference, so an actual attacker must distinguish likely hits from misses statistically.

6.3 Trace-driven attacks: observing the sequence

A trace-driven attacker obtains observations during execution, giving more information than total time or a final footprint. A concurrent program sharing the cache can, under suitable scheduling and measurement conditions, observe activity at individual stages of the victim’s computation. Consider these two access sequences:

```
secret true:    M[4096], M[8192],    M[8192 + 4]
secret false:   M[4096], M[4096+4], M[8192]
```

Starting with the blocks absent, the first sequence is miss–miss–hit, while the second is miss–hit–miss. Both contain two misses and one hit, and both touch the same two memory blocks. In the direct-mapped model, both executions end with the block starting at 8192 in slot 0; if the two blocks can coexist, both executions leave both blocks cached. Thus neither the miss count nor the final footprint distinguishes these branches, while the ordered trace does.

The strongest form of this observation assumes that the attacker can run concurrently with the victim, or can be interleaved with the victim at very fine granularity. In that worst case, the attacker can perform a small measurement between victim accesses: prime or probe a relevant cache set, let the victim execute the next memory operation, and then measure again. Repeating this procedure turns one aggregate footprint into an ordered sequence of cache events. The attacker still does not read the victim’s memory contents, but the timing of the attacker’s own probes reveals when particular cache resources were used.

6.4 Practical limits

Practical attacks face noise from other processes, scheduling, and unrelated victim accesses. Multiple operations may touch the same line, obscuring which operation caused an observation. Statistical aggregation over many executions can nevertheless expose a stable dependency. Noise makes measurement harder; it does not establish noninterference.

7 An Access-Driven Attack on Table-Based AES

7.1 Why the final round matters

AES encrypts 128-bit blocks. AES-128 uses a 128-bit key and ten rounds, with round keys derived by key expansion. Its state can be viewed as a 4×4 byte matrix. The final round applies SubBytes, ShiftRows, and AddRoundKey, omitting MixColumns. Consequently, each ciphertext byte depends on one substituted state byte and one final-round key byte.

Writing π for the byte permutation induced by ShiftRows,

$$C[j] = T(B[\pi(j)]) \oplus K[j].$$

Here B is the internal state entering the final round, T is the S-box permutation, and K denotes the *final-round* key. Focusing on one byte,

$$C = T[i] \oplus K \implies K = C \oplus T[i].$$

If a table lookup implements T , learning the accessed index i would immediately reveal that key byte. Learning only the cache line narrows the possible indices and therefore narrows the possible key bytes. The attack exploits a table-based implementation, not a failure of AES’s mathematical definition. The internal state B is not generally known or directly chosen by the attacker. Chosen external inputs influence it. The attacker seeks to recover the key from the observable ciphertext and cache behavior.

7.2 The attacker and the observations

Assume the attacker knows the table T , observes ciphertexts, and shares cache resources with the victim. Repeated encryptions use a fixed key. The attacker can influence the external input, although this does not give direct control or knowledge of the internal state entering the final round.

A cache observation identifies a line or a set of lines, rather than a precise S-box index for each output byte. Several indices occupy the same line, and multiple lookups can contribute to the same footprint. For one output byte, the candidate table entries imply candidate keys through $K = C \oplus T[i]$. Combining observations can discard candidates that are inconsistent across executions.

This converts an apparently indirect observation about memory use into a constraint on cryptographic key material. Its effectiveness depends on the implementation, cache geometry, and measurement quality. A simpler additive example makes the candidate-filtering mechanism explicit.

8 Worked Toy Example: Narrowing a Secret with Cache Lines

Candidate filtering can be understood through a simpler transformation:

$$C[j] = M[P[j]] + K[j], \quad j \in \{0, 1, 2, 3\}.$$

The attacker knows the output C , the table M , and the set of cache lines accessed by each execution. The input P and key K are hidden.

8.1 Layout and arithmetic assumptions

For this example, assume a table with **eight-byte entries**. With 256 entries this occupies 2048 bytes, or 32 aligned cache lines of 64 bytes. Each line ℓ contains indices

$$I_\ell = \{8\ell, 8\ell + 1, \dots, 8\ell + 7\}.$$

Set $M[i] = i$ and use ordinary integer addition and subtraction.

8.2 Use one fixed output position

Suppose the attacker collects the following observations:

Ciphertext tuple	Observed lines
(0, 1, 0, 16)	{0, 2, 7}
(0, 7, 4, 32)	{0, 2, 4, 9}
(1, 7, 32, 6)	{0, 4, 7, 9}
(1, 0, 0, 16)	{0, 2, 8}

We focus on recovery the first byte (the reasoning for other bytes is the same since they are encrypted independently), $j = 0$. The first two observations have $C[0] = 0$. Notice that since both ciphertexts have first byte = 0, and we used the same key, from $C[0] = M[P[0]] + K[0]$ we can infer that for both observations $P[0]$ is the same. Therefore the two encryption must have accessed the same cache line while encrypting the first byte. Therefore, we intersect their footprints, which gives $\{0, 2\}$. The last two have $C[0] = 1$, have intersection $\{0\}$. Therefore the possible table values satisfy

$$\begin{aligned} A_0 &= \{0, 1, \dots, 7\} \cup \{16, 17, \dots, 23\}, \\ A_1 &= \{0, 1, \dots, 7\}. \end{aligned}$$

It is essential to fix the byte position. Different positions use different $K[j]$, so observing a zero elsewhere in a tuple does not belong in this group.

More generally, for two executions at a fixed position and with the same key,

$$C_1 = M[P_1] + K, \quad C_2 = M[P_2] + K,$$

so subtracting the equations removes the unknown key:

$$C_1 - C_2 = M[P_1] - M[P_2].$$

This relation lets the attacker compare candidate table values without already knowing K .

Let $a_0 \in A_0$ and $a_1 \in A_1$ be the true values for outputs zero and one. Since the key is unchanged,

$$0 = a_0 + K[0], \quad 1 = a_1 + K[0] \implies a_1 - a_0 = 1.$$

Values 16 through 23 cannot satisfy this relation with $a_1 \leq 7$. Also, $a_0 = 7$ would require $a_1 = 8$. Filtering therefore leaves

$$A'_0 = \{0, \dots, 6\}, \quad A'_1 = \{1, \dots, 7\}.$$

The filtering has narrowed, but not uniquely recovered, the key: ordinary integer arithmetic leaves $K[0] \in \{0, -1, \dots, -6\}$. Under modulo-256 arithmetic these correspond to $\{0, 255, 254, 253, 252, 251, 250\}$. Further suitable observations are needed. However we have substantially reduced the space for a brute-force attack. The AES attack uses the same general idea with XOR and the real S-box.

9 Countermeasures and Their Limits

9.1 Remove or isolate the shared resource

Disabling caches removes the cache mechanism under consideration but has a large performance cost. Selectively making victim or attacker accesses uncached requires hardware and system support; its effect depends on which cache levels and attack paths remain available.

Cache partitioning, sometimes implemented through page colouring, assigns memory so different security domains map to disjoint cache resources. For a simplified 8 KiB cache with 4 KiB pages, even-numbered physical pages can be allocated to one process and odd-numbered pages to another. Address 8192+100 is in an even-numbered page; 4096+100 is in an odd-numbered page. This illustrates the allocation principle, assuming the cache index uses the corresponding physical-address bits.

Partitioning can prevent the contention used by access-driven and trace-driven attacks on that cache. It can also waste capacity and become difficult to manage with many processes. It is not automatically isolation of every cache level or every other shared resource.

9.2 Flush or normalize cache state

Flushing at an appropriate entry boundary removes attacker-prepared cache state; flushing at exit removes residual state that could be probed afterwards. Shared-memory attacks can require attention to both boundaries. These measures primarily address observations across execution boundaries. An attacker able to observe concurrently may see the secret-dependent activity before the cleanup.

The victim can also evict sensitive table data on exit or deliberately access all potentially sensitive lines to normalize its footprint. Such measures need the relevant cache effects to be independent of the secret, including effects on other levels and the timing of the cleanup itself.

For total-time channels, delaying the response until a fixed worst-case execution time can conceal a difference in response time, provided that bound is valid and actually enforced. It does not by itself equalize cache activity, power, or intermediate observations.

9.3 Reduce the quality of observations

Noise can be added through random memory accesses or random computation. Timers can be made less precise. Both can increase the number of samples an attacker needs, but repeated measurements can average out noise. External timing sources may also remain available.

Randomization can change a structure's base address, permute its physical layout, or change the order in which independent output bytes are processed. A layout permutation must preserve the program's logical lookup result through an appropriate index mapping. The security benefit depends on what is hidden, how often it changes, and whether repeated observations allow it to be inferred.

9.4 Detect suspicious activity

Detection can look for cache contents consistent with priming and for frequent timing requests consistent with probing. These are detection heuristics: legitimate workloads can exhibit similar behavior, and some clocks can be read without a system call. Detection can complement preventive controls but does not make a secret-dependent implementation noninterfering.

9.5 Constant-time programming

Constant-time programming aims to make relevant execution behavior independent of secrets, while allowing dependence on public values such as an input length that the policy already reveals. Two central rules are:

- do not branch on secrets;
- do not use secrets to determine memory addresses, including table indices.

A fixed instruction count alone is insufficient if the instructions access different cache lines or have secret-dependent latency. Compiled machine code and the arithmetic operations used must also satisfy the intended model. Constant-time behavior for timing/cache observations does not automatically mean constant power consumption or eliminate every physical channel.

For the PIN example, an illustrative fixed-length comparison accumulates all mismatches before returning:

```
diff = 0
for i in 0..3:
    diff = diff OR (candidate[i] XOR stored_pin[i])
return diff == 0
```

All four positions are read for every attempt. This is an explanatory pattern, not a substitute for checking how a particular language and compiler implement it.

For exponentiation, the conditional multiplication can be replaced with

$$b = e \bmod 2, \quad V = b \cdot \text{base} + 1 - b, \quad \text{result} \leftarrow (\text{result} \cdot V) \bmod m.$$

If $b = 0$, then $V = 1$; if $b = 1$, then $V = \text{base}$. The multiplication now appears on every iteration. This removes the explicit secret-dependent branch, but is only a partial repair: the loop still stops at the exponent's bit length, and multiplication or reduction may depend on operand values. A complete design needs a public fixed iteration count and suitable arithmetic as well.

10 Exercises

10.1 Exercise 1

Consider the following early-exit checker, which compares a four-digit decimal PIN from left to right and returns false at the first mismatch:

```
check_pin(candidate):  
    for i in 0..3:  
        if candidate[i] != stored_pin[i]:  
            return False  
    return True
```

The stored PIN remains fixed. Assume an ideal, noiseless timing model

$$T = 100 + 10N,$$

where N is the number of digit comparisons performed and time is measured in arbitrary units. All other work is included in the fixed cost of 100 units. An attacker obtains these observations:

Candidate	Execution time	Returned value
3000	110	False
7000	130	False

Determine everything these observations reveal about the stored PIN. How many possible PINs remain? Explain why the timing reveals more than the two returned values, and state how the conclusion changes if measurements are noisy.

Correct solution. The first call performs $(110 - 100)/10 = 1$ comparison. It therefore fails at the first position, so the first digit is not 3. The second call performs $(130 - 100)/10 = 3$ comparisons and returns false. It must have passed the first two comparisons and failed the third. Consequently, the stored PIN has the form

$$70xy, \quad x \in \{1, 2, \dots, 9\}, \quad y \in \{0, 1, \dots, 9\}.$$

The first digit is 7, the second is 0, and the third is not 0. The fourth digit was never examined, so the observations give no information about it. There are $9 \cdot 10 = 90$ remaining PINs, including leading or trailing zeroes where allowed by these constraints. The first observation adds no further restriction once the first digit has been identified as 7.

The returned values alone only exclude the two complete candidates 3000 and 7000, leaving 9,998 possibilities. The timing exposes where rejection occurred and reduces the candidate set much further. With noise, a single duration cannot be converted reliably into an exact comparison count. Repeated measurements and a calibrated timing model are needed before drawing the same conclusions with confidence.

Wrong solution. “The call with 7000 took three comparisons, so its first three digits are correct. Since the call failed, only the final digit is wrong. The PIN is $700y$ with $y \neq 0$, leaving nine possibilities.”

Why it is wrong. The comparison that detects a mismatch also consumes time. Three comparisons followed by rejection mean two successful comparisons and one failed comparison, not three successful comparisons. If the first three digits were correct, the checker would reach the fourth position and take 140 units in the given model. The wrong solution confuses digits tested with digits matched.

10.2 Exercise 2: Recovering a key from cache-line sets

Consider one fixed output position of the additive transformation

$$C = M[P] + K, \quad M[i] = i.$$

The secret integer key K is unchanged across all executions. Use ordinary integer arithmetic, without wraparound. The table contains 256 eight-byte entries and starts on a 64-byte cache-line boundary. Table line ℓ therefore contains indices 8ℓ through $8\ell + 7$.

For each execution, the attacker observes C and a set E containing every table line accessed. The set includes the line used by this output position, but may also include lines accessed by other operations. Assume there are no missed accesses. Four observations are available:

Execution	C	Observed set E
1	42	$\{0, 2\}$
2	42	$\{0, 3\}$
3	49	$\{0, 4\}$
4	49	$\{0, 5\}$

Recover K and the table index used for each distinct output value. Explain why observing a cache line does not immediately identify a table entry.

Correct solution. Because K is fixed and M is the identity function, repeated occurrences of the same C at this position use the same index $P = C - K$. Intersecting the footprints for $C = 42$ gives $\{0, 2\} \cap \{0, 3\} = \{0\}$. Likewise, for $C = 49$ the intersection is $\{0, 4\} \cap \{0, 5\} = \{0\}$. Both output values therefore use entries in line 0, whose possible indices are $\{0, 1, \dots, 7\}$.

The first group yields the candidate keys

$$\mathcal{K}_{42} = \{42 - p : p \in \{0, \dots, 7\}\} = \{35, 36, \dots, 42\}.$$

The second group yields

$$\mathcal{K}_{49} = \{49 - p : p \in \{0, \dots, 7\}\} = \{42, 43, \dots, 49\}.$$

The key must satisfy both groups, so

$$K \in \mathcal{K}_{42} \cap \mathcal{K}_{49} = \{42\}.$$

Thus $K = 42$. Executions 1 and 2 use $P = 42 - 42 = 0$, while executions 3 and 4 use $P = 49 - 42 = 7$. Both indices lie in line 0, so the result is consistent with all four observations. Equivalently, subtracting the output equations gives $49 - 42 = P_{49} - P_{42} = 7$; within the range 0 through 7, the only possible pair is $P_{49} = 7$ and $P_{42} = 0$.

Wrong solution. “Both intersections contain line 0, so both executions accessed table entry 0. The output 42 therefore implies $K = 42$, while output 49 implies $K = 49$. The observations are contradictory, and no fixed key can explain them.”

Why it is wrong. A cache-line number identifies a block of entries, not one entry. Line 0 holds eight possible indices. The two output groups can use different indices in that same line. Treating the line number as the exact table index discards valid possibilities and creates a contradiction that is absent from the data.